Proyecto Final: Segundo Entregable

Modelos de Machine Learning para Clasificación y Regresión

Pregrado en Ciencia de Datos

Guía Técnica y Entregables

---

1. Introducción

Este entregable tiene como objetivo profundizar en el desarrollo de modelos de **Machine Learning**, incorporando técnicas de optimización de hiperparámetros y validación estadística.

Se espera que el estudiante implemente soluciones con:

- **Rigor estadístico**
- **Eficiencia computacional**
- **Interpretabilidad**
- **Reproducibilidad (control de semillas, versionado, pipelines)**

Sugerencia: estructurar el proyecto como un pipeline reproducible usando `scikit-learn` `Pipeline` o herramientas similares.

2. Panorama General: ¿Cuántos Modelos se Deben Entrenar?

Antes de entrar en el detalle técnico de cada componente (balanceo, optimización de hiperparámetros, optimización computacional, evaluación), es fundamental que el estudiante comprenda que el proyecto **no consiste en entrenar un único modelo**, sino en construir

un **pipeline combinatorio**: cada modelo base debe combinarse con cada técnica aplicable de las etapas siguientes, y el resultado final es el producto (no la suma) de todas esas alternativas.

Regla de oro: si en la Etapa 1 hay m modelos, en la Etapa 2 hay b técnicas de balanceo y en la Etapa 3 hay o métodos de optimización, entonces el número total de modelos entrenados y evaluados es $m \times b \times o$, **no** $m + b + o$. El balanceo de clases únicamente tiene sentido para problemas de **clasificación**; en **regresión** esta etapa se omite.

La Figura 1 ilustra este proceso paso a paso, separando la rama de clasificación (que sí incluye balanceo) de la rama de regresión (que no lo incluye), y mostrando cómo se acumula la multiplicación hasta llegar al número final de modelos que cada estudiante debe entrenar, comparar y reportar.

Cuadro 1: Desglose aritmético de la explosión combinatoria mostrada en la Figura 1

Tarea	Modelos	Balanceo	Optimización	Operación	Total
Clasificación	7	4	4	$7 \times 4 \times 4$	**112**
Regresión	7	— (no aplica)	4	7×4	**28**
Total general de modelos entrenados y evaluados					**140**

Implicaciones prácticas de esta explosión combinatoria:

- **Automatización obligatoria:** con 140 combinaciones, entrenar modelo por modelo de forma manual es inviable. El pipeline debe programarse mediante bucles anidados o funciones parametrizadas (modelo → balanceo → optimizador).
- **Costo computacional:** cada una de las 140 combinaciones implica, a su vez, múltiples iteraciones internas de validación cruzada dentro del propio proceso de optimización (Grid, Random, Bayesiano o Genético). Es indispensable priorizar las técnicas de la Sección 4 (Optimización Computacional).
- **Organización de resultados:** se recomienda registrar cada corrida (modelo, técnica de balanceo, método de optimización, métricas, tiempo de ejecución) en una tabla o *dataframe* maestro, para poder filtrar, comparar y graficar resultados sin perder trazabilidad.
- **No todas las combinaciones son igual de relevantes:** el estudiante debe usar criterio para reportar de forma exhaustiva las combinaciones más informativas, sin sacrificar el rigor comparativo exigido en las Secciones 5 y 6.

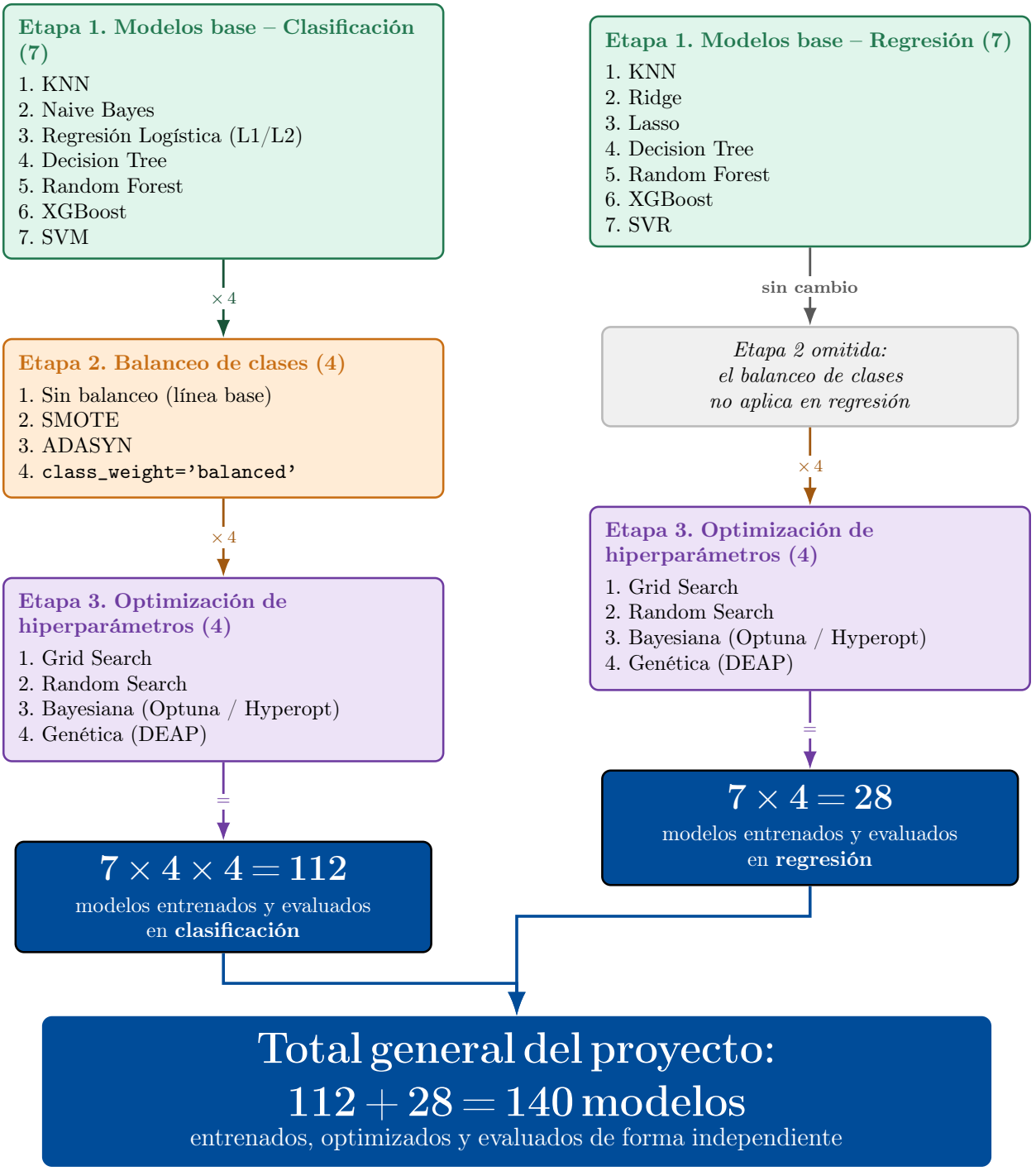


Figura 1: Explosión combinatoria del pipeline: cada modelo base se combina con cada técnica de balanceo aplicable (solo en clasificación) y, a su vez, cada una de esas combinaciones se ajusta con cada método de optimización de hiperparámetros exigido. El resultado no es una suma de técnicas, sino un producto de alternativas.

3. Optimización de Hiperparámetros

3.1. Objetivo

Optimizar el desempeño de los modelos mediante técnicas de búsqueda eficiente en espacios de alta dimensión, sustentando cada decisión metodológica en la teoría subyacente del método empleado y no únicamente en su implementación de librería.

3.2. Validación Anidada (Nested Cross-Validation)

Dado que el proceso de optimización de hiperparámetros implica, en sí mismo, una búsqueda basada en el desempeño observado en los datos, reportar la métrica final del mejor modelo directamente sobre el mismo esquema de validación utilizado para seleccionarlo produce una estimación optimista (sesgo de selección de modelo). Para evitar este problema, es obligatorio implementar un esquema de **validación cruzada anidada**:

- **Bucle externo (outer loop):** particiona los datos en K pliegues y se utiliza exclusivamente para estimar el desempeño de generalización del modelo ya optimizado, de forma completamente independiente del proceso de búsqueda.
- **Bucle interno (inner loop):** dentro de cada partición de entrenamiento del bucle externo, se ejecuta el proceso completo de optimización de hiperparámetros (Grid, Random, Bayesiano o Genético) usando su propio esquema de validación cruzada.

La métrica de desempeño que se reporta en las Secciones 5 y 6 debe provenir del bucle externo, nunca del valor óptimo hallado dentro del bucle interno. Confundir ambos niveles es una de las fuentes más comunes de sobreestimación del desempeño en proyectos de Machine Learning.

3.3. Métodos requeridos

- **Grid Search.**
- **Random Search.**
- **Optimización Bayesiana (ej: Optuna o Hyperopt)**. Es obligatorio implementar manualmente un pipeline completo (preprocesamiento + modelo) dentro de cada iteración de validación, garantizando la ausencia de **data leakage**, dado que estas librerías no integran esta funcionalidad de forma nativa. Adicionalmente, se debe reportar y justificar de forma explícita:

  - El **modelo sustituto o *surrogate model*** empleado (proceso gaussiano o *Tree-structured Parzen Estimator*, según la librería).
  - La **función de adquisición** utilizada (Mejora Esperada / *Expected Improvement*, Límite Superior de Confianza / *Upper Confidence Bound*, o Probabilidad de Mejora / *Probability of Improvement*) y el balance exploración-explotación que dicha función privilegia.

- **Algoritmos Genéticos (ej: DEAP)**. Se debe implementar manualmente un pipeline completo dentro del proceso evolutivo, asegurando que cada evaluación del individuo incluya el preprocesamiento ajustado únicamente sobre los datos de entrenamiento, evitando **data leakage**. Se debe especificar y justificar explícitamente:
  - El operador de **selección** (torneo, ruleta, ranking).
  - Los operadores de **cruce** y **mutación**, junto con sus probabilidades asociadas.
  - La estrategia de **elitismo** y el tamaño de la población.
  - La evolución de la **diversidad genética** de la población a lo largo de las generaciones, como indicador de convergencia prematura.
- **Optimización multi-fidelidad (*Successive Halving* / Hyperband / BOHB).** Como técnica adicional de eficiencia, se recomienda implementar al menos un método de eliminación sucesiva de configuraciones poco prometedoras usando presupuestos parciales (menos árboles, menos épocas, submuestras de menor tamaño), reasignando el presupuesto computacional ahorrado a las configuraciones más competitivas. Se debe justificar la métrica de fidelidad utilizada y el factor de reducción entre rondas.
- **Nota general:** Para todos los métodos de optimización (incluyendo Grid Search y Random Search), se debe garantizar que el preprocesamiento esté correctamente encapsulado dentro del proceso de validación (ya sea mediante `Pipeline` de `scikit-learn` o implementación manual equivalente), evitando cualquier fuga de información entre entrenamiento y prueba.

3.4. Requerimientos

- Definir espacios de búsqueda realistas (evitar rangos arbitrarios).
- Justificar la elección de hiperparámetros.

- Implementar y justificar un esquema de validación anidada (Sección 3.2), seleccionando el número de pliegues externos e internos que mejor se ajuste a la estructura y al tamaño de los datos.
- Comparar métodos usando:
  - Métricas del modelo (obtenidas del bucle externo).
  - Tiempo de ejecución total y por evaluación individual (*wall-clock time*).
  - Número de iteraciones/evaluaciones y presupuesto computacional total consumido.
- Analizar convergencia (especialmente en Bayesiano y Genético) mediante curvas de **desempeño en cualquier momento** (*anytime performance*): mejor valor de la métrica objetivo encontrado hasta la evaluación t, en función de t.

Sugerencias prácticas:

- Usar escalas logarítmicas para hiperparámetros como C, λ, learning rate.
- Visualizar el proceso de optimización (ej: history plots en Optuna, superficies de la función sustituta).
- Comparar estabilidad de soluciones encontradas ante distintas semillas aleatorias.

Se debe demostrar experimentalmente, con evidencia cuantitativa y no solo apreciación visual, por qué métodos como **Bayesiano o Genético** pueden ser más eficientes que Grid/Random Search en términos de calidad de solución alcanzada por unidad de presupuesto computacional.

4. Optimización Computacional

4.1. Objetivo

Reducir el costo computacional manteniendo o mejorando el desempeño del modelo, sustentando cada mejora en un análisis formal de complejidad y no únicamente en la medición empírica de tiempos.

4.2. Análisis de Complejidad Algorítmica

Para cada modelo optimizado en esta sección, se debe reportar de forma explícita su **complejidad asintótica** en notación $O(\cdot)$, tanto para la fase de entrenamiento como para la fase de inferencia, en función del número de observaciones n, del número de características p y, cuando aplique, del número de árboles o vecinos considerados. Este análisis teórico debe contrastarse directamente con el tiempo empírico medido, discutiendo si el comportamiento observado es consistente con el orden de complejidad esperado.

- **KNN:** búsqueda exhaustiva $O(n \cdot p)$ por consulta frente a estructuras de indexación como KD-Trees o Ball Trees ($O(\log n)$ en bajas dimensiones) y búsqueda aproximada de vecinos más cercanos mediante FAISS (*Approximate Nearest Neighbors*), discutiendo el compromiso entre exactitud y velocidad que introduce la aproximación.
- **Ridge/Lasso:** solver SAGA, con complejidad por iteración lineal en n y p, adecuado para problemas de gran escala y matrices dispersas.
- **Naive Bayes:** `partial_fit` para aprendizaje incremental (*online learning*) cuando el conjunto de datos no cabe en memoria.
- **XGBoost:** método `hist` (histogramas de características) frente al método exacto, y *early stopping* basado en un conjunto de validación independiente.
- **SVM:** descenso de gradiente estocástico (SGD) y `LinearSVC`, evitando la complejidad cuadrática o cúbica del núcleo completo en problemas de gran tamaño de muestra.

4.3. Paralelización y Cómputo Distribuido

Dado el volumen total de modelos que deben entrenarse (véase la Sección 2), se requiere aprovechar el paralelismo disponible:

- Uso de `n_jobs` y *backends* de `joblib` (`loky`, `threading`) para paralelizar entre modelos y entre pliegues de validación cruzada.
- Consideración de herramientas de cómputo distribuido (Dask, Ray) cuando el número de combinaciones y el tamaño de los datos lo justifiquen.
- Uso de aceleración por GPU cuando esté disponible (XGBoost `device='cuda'`), reportando el factor de aceleración obtenido frente a la ejecución en CPU.

4.4. Perfilamiento (*Profiling*)

Se recomienda perfilar el uso de tiempo de CPU y de memoria (`cProfile`, `memory_profiler` o `py-spy`) para identificar los cuellos de botella reales del pipeline antes de optimizar prematuramente componentes que no dominan el costo total.

Sugerencias prácticas:

- Medir tiempo con `time` o `timeit`, promediando sobre varias repeticiones para reducir la varianza de la medición.
- Comparar complejidad teórica vs tiempo empírico, idealmente variando n y p de forma controlada para estimar el exponente empírico de la curva de escalamiento.
- Evaluar el compromiso (*trade-off*) entre precisión y velocidad, y entre velocidad y consumo de memoria.

Se debe presentar una tabla comparativa clara (modelo estándar vs optimizado) que incluya: complejidad teórica, tiempo de entrenamiento, tiempo de inferencia, consumo de memoria y variación en el desempeño predictivo.

5. Evaluación de Modelos

5.1. Clasificación

5.1.1. Métricas de Evaluación

Para cada modelo de clasificación implementado, se debe presentar:

- Matriz de confusión
- Curva ROC (Receiver Operating Characteristic)
- Tabla de métricas de precisión que incluya: Accuracy, Precision, Recall, F1-score y AUC

Justificación de la métrica de validación: Se debe explicar explícitamente qué métrica fue seleccionada como criterio principal de validación y por qué. En problemas con clases desbalanceadas, se debe argumentar la elección de métricas como *recall* (sensibilidad) cuando el costo de los falsos negativos es elevado, o *precision* cuando los falsos positivos son más críticos.

Ejemplo de presentación de resultados:

Modelo	Accuracy	Precision	Recall	F1-score	AUC
Logistic Regression	0.87	0.85	0.89	0.87	0.91
Random Forest	0.90	0.88	0.92	0.90	0.94
XGBoost	0.91	0.89	0.93	0.91	0.95

Cuadro 2: Comparación de métricas de clasificación en el conjunto de prueba

5.1.2. Balanceo de Clases

En problemas de clasificación con desbalanceo de clases, se requiere la implementación y comparación de técnicas de balanceo. Se deben definir e implementar explícitamente las siguientes estrategias, justificando su selección en el contexto del problema:

- **SMOTE (Synthetic Minority Over-sampling Technique):** Generación de instancias sintéticas de la clase minoritaria mediante interpolación entre ejemplos cercanos en el espacio de características.
- **ADASYN (Adaptive Synthetic Sampling):** Enfoque adaptativo que genera un mayor número de muestras sintéticas para aquellas instancias de la clase minoritaria que son más difíciles de aprender, utilizando una distribución de densidad.
- **class_weight='balanced':** Ajuste automático de los pesos de las clases en el algoritmo de aprendizaje para penalizar de manera diferencial los errores de clasificación, asignando mayor peso a la clase minoritaria.

El análisis debe incluir una comparación del rendimiento de los modelos con y sin la aplicación de estas técnicas, evaluando métricas apropiadas para contextos desbalanceados como el *recall*, el *F1-score* o el área bajo la curva ROC (AUC).

5.1.3. Calibración de Probabilidades

Un modelo con alto poder discriminativo (AUC elevado) no necesariamente produce probabilidades bien calibradas, es decir, probabilidades que reflejen la frecuencia real de ocurrencia del evento. Para cada modelo de clasificación se debe evaluar su calibración mediante:

- **Diagrama de confiabilidad (*reliability diagram*):** probabilidad predicha vs frecuencia observada, agrupando las predicciones en *bins*.
- **Brier Score:** error cuadrático medio entre las probabilidades predichas y los desenlaces observados, como métrica global de calibración y discriminación conjunta.

- **Error de Calibración Esperado (*Expected Calibration Error*, ECE):** promedio ponderado de la discrepancia entre confianza y precisión a través de los *bins*.

Cuando se detecte una calibración deficiente (frecuente en SVM sin ajuste adicional o en modelos de árboles con probabilidades extremas), se debe discutir y, si es pertinente, aplicar una técnica de recalibración (escalado de Platt o regresión isotónica), reportando su efecto sobre el ECE y el Brier Score.

5.2. Regresión

5.2.1. Métricas de Evaluación

Para cada modelo de regresión implementado, se debe presentar:

- RMSE (Root Mean Square Error)
- MAE (Mean Absolute Error)
- Incluir una representación gráfica que muestre los conjuntos de entrenamiento, validación y prueba, junto con las respectivas predicciones del modelo, permitiendo evaluar visualmente el ajuste en cada subconjunto.

Ejemplo de presentación de resultados:

Modelo	RMSE	MAE	p-valor (White)	p-valor (BDS)
Ridge Regression	0.124	0.098	0.342	0.107
Random Forest	0.102	0.081	0.581	0.234
XGBoost	0.095	0.076	0.623	0.418

Cuadro 3: Comparación de métricas de regresión y pruebas de supuestos en el conjunto de prueba

5.2.2. Análisis de Residuos

El análisis de residuos constituye una herramienta fundamental para validar los supuestos del modelo de regresión. Se deben abordar los siguientes aspectos:

- Verificar supuestos:
  - **Homocedasticidad:** Aplicar el test de White para evaluar la constancia de la varianza de los errores.

    - **Independencia:** Aplicar el test de BDS para evaluar la independencia de los residuos.

- Identificar patrones en residuos que puedan indicar un modelo mal especificado (ej., formas sistemáticas, heterocedasticidad).
- Presentar la función de autocorrelación (**ACF**) de los residuos, acompañada del valor p del test de **Ljung-Box** para evaluar la independencia serial y detectar posibles estructuras de dependencia remanente.
- Incluir un histograma de los residuos para verificar visualmente el supuesto de normalidad, complementado con pruebas de normalidad cuando sea pertinente.

5.3. Interpretabilidad Global y Local

Para el mejor modelo identificado en cada tarea (clasificación y regresión), se debe realizar un análisis de interpretabilidad basado en **valores de Shapley (SHAP)**, fundamentado en la teoría de juegos cooperativos, que satisface las propiedades de eficiencia, simetría y aditividad en la atribución de la contribución de cada variable a la predicción. El análisis debe incluir, como mínimo:

- **Interpretabilidad global:** gráfico resumen (*summary plot*) que combine la magnitud e importancia relativa de cada característica sobre el conjunto de datos completo.
- **Interpretabilidad local:** explicación tipo *waterfall* o *force plot* para al menos dos observaciones individuales (una correctamente clasificada/predicha y una con error elevado), discutiendo qué variables explican dicha diferencia.
- **Comparación con LIME (Sección 5, XGBoost):** discutir las diferencias metodológicas entre una explicación basada en aproximaciones locales lineales (LIME) y una basada en valores de Shapley exactos o aproximados (SHAP), y en qué escenarios los resultados de ambas técnicas podrían divergir.

5.4. Robustez y Estabilidad de las Métricas

Reportar una métrica como un único número (por ejemplo, "AUC = 0.91") oculta la variabilidad inherente al proceso de muestreo y de optimización. Para cada una de las combinaciones reportadas se debe presentar:

- La métrica de interés como **media ± desviación estándar** a través de los pliegues del bucle externo de validación anidada (Sección 3.2), y no como un valor puntual de una única partición.

- Un análisis de sensibilidad frente a distintas semillas aleatorias de inicialización, cuando el algoritmo lo permita (Random Forest, XGBoost, algoritmos genéticos, inicialización de SVM/redes).
- Una discusión explícita sobre si las diferencias observadas entre modelos son mayores que la variabilidad intrínseca de cada modelo individual, lo cual motiva directamente el uso de pruebas estadísticas formales en la Sección 6.

6. Comparación Estadística

Para garantizar la validez de las comparaciones entre modelos, se deben implementar las siguientes pruebas estadísticas.

Un problema de escala: como se estableció en la Sección 2, el proyecto genera 140 modelos. El número de comparaciones pareadas posibles entre ellos es $\binom{140}{2} = 9730$. Ejecutar 9730 pruebas de hipótesis independientes sin ningún control, usando un umbral $\alpha = 0{,}05$, produciría en promedio cerca de 487 falsos positivos solo por azar (inflación del error de tipo I). Por esta razón, la comparación estadística **no puede reducirse** a aplicar DeLong o Diebold-Mariano a todos los pares de modelos; se requiere una estrategia jerárquica de dos etapas.

6.1. Etapa 1: Prueba Global (Ómnibus)

Antes de realizar cualquier comparación pareada, se debe establecer si existe evidencia de que **al menos un modelo** difiere significativamente de los demás, usando una prueba no paramétrica de medidas repetidas sobre los pliegues de validación cruzada:

- **Test de Friedman:** análogo no paramétrico del ANOVA de medidas repetidas, aplicado sobre el ranking de cada modelo en cada pliegue de validación cruzada. La hipótesis nula establece que todos los modelos tienen un rendimiento equivalente (rangos promedio iguales).
- Si el test de Friedman resulta significativo, se procede al análisis *post-hoc*; en caso contrario, no existe evidencia estadística suficiente para afirmar que algún modelo supera a los demás, independientemente de las diferencias observadas en las métricas puntuales.

6.2. Etapa 2: Análisis *Post-Hoc* y Comparaciones Dirigidas

- **Test de Nemenyi:** prueba *post-hoc* que compara todos los modelos entre sí controlando el error de tipo I de forma conjunta (familywise), calculando una **Diferencia Crítica (CD)** por debajo de la cual dos modelos se consideran estadísticamente equivalentes.
- **Diagrama de Diferencia Crítica (CD Diagram):** representación gráfica obligatoria que ubica cada modelo según su rango promedio y conecta con una línea horizontal a aquellos modelos cuya diferencia no es estadísticamente significativa según Nemenyi. Este diagrama sustituye, de forma compacta y rigurosa, a una tabla de 9730 comparaciones.
- **Test de DeLong:** reservado para comparar curvas ROC y valores de AUC entre un subconjunto reducido y justificado de modelos de clasificación (por ejemplo, los mejores dos o tres modelos por familia identificados en la Etapa 1).
- **Test de Diebold-Mariano:** reservado, de forma análoga, para comparar la precisión predictiva entre un subconjunto reducido de modelos de regresión, con la corrección de Harvey-Leybourne-Newbold para muestras finitas. Dado el volumen de modelos de regresión del proyecto, esta prueba debe usarse como confirmación puntual dentro de la secuencia jerárquica más amplia descrita en la Sección 6.3, y no de forma aislada.
- **Corrección por comparaciones múltiples:** cuando se reporte un conjunto de comparaciones dirigidas (DeLong o Diebold-Mariano) adicionales a Friedman-Nemenyi, los valores p deben ajustarse mediante el método de Holm-Bonferroni o mediante control de la tasa de falsos descubrimientos (Benjamini-Hochberg), reportando tanto el p-valor crudo como el ajustado.
- **Bootstrap:** para estimar intervalos de confianza de las métricas de rendimiento y evaluar la estabilidad de las estimaciones, preferiblemente mediante el método *bias-corrected and accelerated* (BCa) cuando la distribución de la métrica sea asimétrica.

6.3. Más Allá de Diebold-Mariano: Secuencia Jerárquica para Modelos de Regresión

El test de Diebold-Mariano es un punto de partida sólido, pero fue diseñado en 1995 para comparar de a pares la precisión predictiva de dos modelos, bajo supuestos que en la práctica moderna suelen violarse:

- **Solo permite comparaciones de a pares.** Con el número de modelos de regresión de este proyecto, el DM por sí solo reproduce el mismo problema de comparaciones

múltiples ya identificado para clasificación: docenas de pares e inflación del error tipo I si no se controla adecuadamente.

- **Supone estacionariedad de la serie de diferencias de pérdida.** Si existen cambios estructurales o el error relativo entre modelos varía en el tiempo (frecuente en series financieras o macroeconómicas), el DM puede no detectar una superioridad que solo existe en ciertos regímenes.

- **Está sesgado en modelos anidados.** Cuando un modelo es caso particular de otro (por ejemplo, Ridge con $\lambda \to 0$ frente a OLS, o un modelo con menos regresores anidado en uno más grande), el estadístico DM tiende a rechazar de más a favor del modelo más grande incluso bajo la hipótesis nula.

- **No corrige por *data snooping*.** Si se prueban muchos modelos y solo se reporta el DM del "ganador" contra el segundo lugar, el resultado queda contaminado por el proceso de selección previo.

Por ello, de forma análoga a la secuencia Friedman → Nemenyi / diagrama de Diferencia Crítica usada en clasificación, los modelos de regresión deben evaluarse mediante las siguientes alternativas modernas, que complementan al DM sin reemplazarlo:

- **Model Confidence Set (MCS)** (Hansen, Lunde y Nason, 2011): en lugar de comparar pares, parte de todo el universo de modelos de regresión y, mediante bootstrap, elimina iterativamente los modelos significativamente peores hasta obtener un "conjunto de confianza" de modelos estadísticamente indistinguibles del mejor. Es el análogo natural, para pronósticos, del diagrama de Diferencia Crítica de Friedman-Nemenyi: resuelve el problema de escala sin necesidad de corrección manual por comparaciones múltiples.

- **Test de Giacomini-White** (2006): generaliza el DM en dos sentidos: (1) permite evaluar habilidad predictiva *condicional* (si un modelo es mejor dado el estado actual del mercado o del ciclo, y no solo en promedio incondicional); (2) es válido incluso cuando los modelos se re-estiman con ventana móvil y hay incertidumbre de estimación de parámetros, algo que el DM clásico ignora.

- **Test de Clark-West** (2007): corrige específicamente el sesgo del DM en modelos anidados, ajustando el estadístico para que la comparación entre un modelo restringido y uno más general sea válida bajo la hipótesis nula de igual capacidad predictiva poblacional.

- **Reality Check** (White, 2000) / **SPA** (Hansen, 2005): pruebas basadas en bootstrap diseñadas para responder si algún modelo de un conjunto grande supera de forma genuina al benchmark, o si la aparente superioridad es producto de haber probado muchos modelos (control de *data snooping*). El SPA de Hansen corrige además un sesgo conservador del Reality Check original.
- **Bootstrap estacionario** (Politis y Romano, 1994): aplicado a la serie de diferencias de pérdida en vez del bootstrap i.i.d. clásico, cuando hay dependencia temporal que el DM asintótico no captura bien en muestras finitas. Complementa, no reemplaza, la corrección HLN ya exigida para el DM.

Secuencia jerárquica recomendada para regresión (análoga a Friedman → Nemenyi en clasificación):

1. **MCS** sobre el universo completo de modelos de regresión, para filtrar a un subconjunto reducido y defendible.
2. **Giacomini-White** o **Clark-West** para las comparaciones dirigidas entre los finalistas resultantes del MCS, según exista o no anidamiento entre ellos.
3. **Diebold-Mariano con corrección HLN** como confirmación puntual entre el mejor modelo y el segundo lugar, complementado con **bootstrap estacionario** para el intervalo de confianza.

Cuadro 4: Ejemplo de resultado del Model Confidence Set (MCS) sobre el universo de modelos de regresión, $\alpha = 0{,}10$

Modelo	Rango promedio de pérdida	p-valor MCS	¿En el conjunto de confianza?
XGBoost	1.3	1.000	Sí
Random Forest	1.9	0.712	Sí
Gradient Boosting	2.4	0.489	Sí
Ridge Regression	4.6	0.038	No
Lasso Regression	5.1	0.021	No

6.4. Tamaño del Efecto

La significancia estadística no implica relevancia práctica, en particular con el elevado poder estadístico que resulta de comparar modelos sobre múltiples pliegues. Todo resultado significativo debe complementarse con una medida de **tamaño de efecto**:

- **Delta de Cliff:** medida no paramétrica de dominancia estocástica, recomendada junto con Friedman-Nemenyi por ser consistente con el uso de rangos.
- d **de Cohen:** para diferencias de pérdida entre modelos de regresión, cuando sea razonable asumir distribuciones aproximadamente simétricas de los errores.

Sugerencias:

- Interpretar los p-values correctamente, no limitándose únicamente a su reporte, sino discutiendo su significado en el contexto del problema.
- Analizar la magnitud del efecto además de la significancia estadística, reportando tamaños de efecto en todos los casos, no solo cuando resulten favorables a la hipótesis del estudiante.
- Reportar intervalos de confianza para las métricas principales, permitiendo una evaluación más completa de la incertidumbre asociada a las estimaciones.

Cuadro 5: Ejemplo de resultado de la Etapa 1 (prueba ómnibus de Friedman) sobre el ranking de los 7 modelos de clasificación a través de los pliegues del bucle externo

Estadístico de Friedman	**g.l.**	p**-valor**	**Conclusión**
$\chi^2_F = 24{,}86$	6	$< 0{,}001$	Se rechaza H_0: existen diferencias significativas

Cuadro 6: Comparación dirigida por pares (subconjunto reducido) de modelos de clasificación mediante el test de DeLong, con corrección de Holm-Bonferroni

Modelo A	**Modelo B**	**Diferencia de AUC**	**IC 95 % Bootstrap**	p**-valor crudo**	p**-valor ajustado**
Logistic Regression	Random Forest	-0.032	[-0.048, -0.016]	0.012	0.024
Logistic Regression	XGBoost	-0.041	[-0.059, -0.023]	0.003	0.009
Random Forest	XGBoost	-0.009	[-0.021, 0.003]	0.214	0.214

Cuadro 7: Comparación dirigida por pares (subconjunto reducido) de modelos de regresión mediante el test de Diebold-Mariano (corrección HLN), con ajuste de Holm-Bonferroni

Modelo A	**Modelo B**	**Diferencia de pérdida (DM)**	**IC 95 % Bootstrap**	p**-valor crudo**	p**-valor ajustado**
Ridge Regression	Random Forest	0.023	[0.009, 0.037]	0.008	0.016
Ridge Regression	XGBoost	0.031	[0.014, 0.048]	0.002	0.006
Random Forest	XGBoost	0.008	[-0.004, 0.020]	0.186	0.186

Cuadro 8: Intervalos de confianza al 95 % para métricas de rendimiento (bootstrap)

Modelo	Métrica	IC 95 % Inferior	IC 95 % Superior
Logistic Regression	AUC	0.872	0.908
Random Forest	AUC	0.911	0.949
XGBoost	AUC	0.918	0.956
Ridge Regression	RMSE	0.112	0.136
Random Forest	RMSE	0.092	0.112
XGBoost	RMSE	0.085	0.105

No basta con decir que un modelo es mejor: debe demostrarse estadísticamente mediante las pruebas correspondientes. Para **clasificación**, la secuencia completa es Friedman → Nemenyi / diagrama de Diferencia Crítica → comparaciones dirigidas (DeLong) con corrección por comparaciones múltiples → tamaño del efecto. Para **regresión**, la secuencia análoga es MCS → Giacomini-White / Clark-West → Diebold-Mariano (HLN) con bootstrap estacionario → tamaño del efecto. Presentar únicamente p-valores individuales de DeLong o Diebold-Mariano sin este contexto jerárquico se considera insuficiente dado el volumen de modelos del proyecto.

7. Entregables

El proyecto debe ser entregado en los siguientes formatos:

- **Jupyter Book (JBook):** Cuaderno interactivo con el código fuente, debidamente comentado y organizado, que permita la reproducción completa de los análisis y resultados presentados. Que integre imágenes, tablas, resultados y su respectiva interpretación. Debe incluir todas las secciones descritas en esta guía.

7.1. Mejoras respecto a la primera entrega

- **EDA mejorado (Sección Data):** Realizar todas las correcciones solicitadas en la primera entrega y mejorar el análisis exploratorio con base en nuevos hallazgos encontrados durante el desarrollo del proyecto.

7.2. Modelos benchmark (Sección Modelos y Resultados)

Se requiere la definición e implementación de los siguientes modelos:

- K-Nearest Neighbors (KNN)

- Clasificación Naïve Bayes
- Regresión Logística con regularización L1 y L2
- Ridge y Lasso
- Decision Tree
- Random Forest
- XGBoost (incluyendo análisis de importancia de características con LIME y con SHAP, Sección 5)
- Support Vector Machine (SVM) y Support Vector Regression (SVR)

7.3. Reproducibilidad e Ingeniería del Pipeline

Dado el volumen de experimentos del proyecto (140 modelos, Sección 2), la calidad de ingeniería del código es tan evaluable como la calidad de los resultados. Se requiere:

- **Entorno fijado:** archivo `requirements.txt` o `environment.yml` con versiones exactas de todas las librerías utilizadas, de forma que el JBook sea reproducible en una máquina distinta.
- **Control de semillas:** una única semilla global documentada, propagada de forma consistente a `numpy`, `scikit-learn`, `XGBoost` y a los algoritmos genéticos.
- **Registro estructurado de experimentos:** cada una de las 140 combinaciones (modelo, técnica de balanceo, método de optimización) debe registrarse en una tabla maestra (*CSV*, *parquet* o herramienta de *experiment tracking* como MLflow) con sus hiperparámetros finales, métricas del bucle externo, tiempo de ejecución y semilla utilizada.
- **Modularidad del código:** separación clara entre módulos de carga y preprocesamiento de datos, definición de modelos, optimización de hiperparámetros y evaluación, evitando bloques de código monolíticos y repetidos entre modelos.
- **Documentación de funciones:** *docstrings* (estilo NumPy o Google) para toda función reutilizada por más de un modelo, describiendo parámetros, valores de retorno y supuestos.

8. Presentación

La sustentación tendrá una duración de **10 minutos** y deberá cubrir los siguientes componentes:

- ETL (Extracción, Transformación y Carga de datos)
- EDA (Análisis Exploratorio de Datos)
- Modelos implementados y resultados obtenidos
- Comparación de métodos de optimización
- Resultados finales y conclusiones

Sugerencias:

- Usar visualizaciones claras (gráficos comparativos) que faciliten la comprensión de los resultados.
- Mostrar evolución de métricas durante la optimización para evidenciar el proceso de mejora.
- Enfatizar decisiones tomadas y su justificación, demostrando pensamiento crítico en la selección de modelos y técnicas.

9. Conclusiones

El estudiante debe demostrar:

- Dominio de técnicas modernas de optimización de hiperparámetros
- Capacidad de análisis comparativo entre modelos y metodologías
- Rigor experimental en la evaluación y validación de modelos
- Pensamiento crítico en la selección de modelos y métricas apropiadas al contexto del problema

Este entregable enfatiza el uso de técnicas utilizadas en entornos reales de Machine Learning, donde la optimización eficiente y la correcta evaluación son clave para el éxito del modelo.